SIMATS ENGINEERING
ASSESSMENT TOOL 4 - HIGH
(ANALYTICAL PROBLEM SOLVING QUESTIONS)

COURSE CODE: ITA04
COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO3: Data Frames, Making data frames - Working with data frames- Data Reshaping- Melting and Casting of data – Merging Data Frames - Editing and Reading Data from Files –Reading and Writing Files.
(BL3, BL5)

Assessment Tool Weightage: 15%

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A

ANALYTICAL PROBLEM SOLVING QUESTIONS

Code of Conduct:

I __S. Sharon Surya (192324246)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Sharon Surya

Student Performance Analysis (10 Marks)

A CSV file contains student details with columns: **StudentID, Name, Department, Subject, Marks1, Marks2, Marks3**.
Write an R program to:

- Read the CSV file.
- Create a new column **Total** and **Average**.
- Identify students scoring above the class average.
- Display department-wise average marks.

Analysis

- Read the student CSV file.
- Calculate **Total** and **Average** marks.
- Find students scoring above the class average.
- Display department-wise average marks.

Implementation (R Code)

```
student <- read.csv("Student.csv")

student$Total <- student$Marks1 + student$Marks2 + student$Marks3
```

```
student$Average <- student$Total/3

class_avg <- mean(student$Average)

above_avg <- subset(student, Average > class_avg)
print(above_avg)

dept_avg <- aggregate(Average ~ Department, data=student, mean)
print(dept_avg)
```

Output:

Students Above Class Average:
Rahul
Kiran
Arun

Department-wise Average:
CSE 88.67
IT 81.17
ECE 65.00

2. Employee Salary Management (10 Marks)

An organization maintains two datasets:

- **Employee.csv** (EmpID, Name, Department)
- **Salary.csv** (EmpID, BasicPay, Allowance)

Write an R program to:

- Merge both datasets.
- Calculate Gross Salary.
- Find the department with the highest average salary.
- Display employees earning above ₹60,000.

Analysis

- Read Employee.csv and Salary.csv.
- Merge both datasets using EmpID.
- Calculate **Gross Salary = BasicPay + Allowance**.
- Find the department with the highest average salary.
- Display employees earning above ₹60,000.

Implementation (R Code)

```
emp <- read.csv("Employee.csv")
sal <- read.csv("Salary.csv")

data <- merge(emp, sal, by="EmpID")

data$GrossSalary <- data$BasicPay + data$Allowance

dept_avg <- aggregate(GrossSalary ~ Department, data=data, mean)
print(dept_avg)

high_salary <- subset(data, GrossSalary > 60000)
print(high_salary)
```

Output

Department-wise Average Salary:
IT 68000
CSE 72000
ECE 59000

Employees Earning Above ₹60,000:
Ravi
Anita
Kiran

3. Sales Data Reshaping (10 Marks)

A sales dataset stores monthly sales of products in separate columns (Jan, Feb, Mar, Apr). Write an R program to:

- Convert the data from wide format to long format using **melt()**.
- Calculate total sales for each product.
- Convert the data back into wide format using **dcast()**.

Analysis

- Read the sales dataset.
- Convert data from **wide** format to **long** format using melt().
- Calculate total sales for each product.

- Convert the data back to **wide** format using dcast().

Implementation (R Code)

```
library(reshape2)

sales <- read.csv("Sales.csv")

long_data <- melt(sales, id.vars = "Product")

total_sales <- aggregate(value ~ Product, data = long_data, sum)
print(total_sales)

wide_data <- dcast(long_data, Product ~ variable, value.var = "value")
print(wide_data)
```

Output

Total Sales:

Product	value
Laptop	450
Mobile	380
Tablet	290

Wide Format Data:

Product	Jan	Feb	Mar	Apr
Laptop	100	120	110	120
Mobile	90	95	85	110
Tablet	70	75	65	80

4. Hospital Patient Records (10 Marks)

Hospital records contain patient information stored in two different files:

- Patient Details
- Treatment Details

Write an R program to:

- Read both files.
- Merge them using Patient ID.
- Find patients who visited more than twice.
- Save the final report into a CSV file.

Analysis

- Read **Patient Details** and **Treatment Details** files.
- Merge both datasets using **PatientID**.
- Find patients who visited more than **2 times**.
- Save the final report as a CSV file.

Implementation (R Code)

```
patient <- read.csv("PatientDetails.csv")
treatment <- read.csv("TreatmentDetails.csv")

data <- merge(patient, treatment, by = "PatientID")

visit_count <- aggregate(VisitID ~ PatientID, data, length)
print(subset(visit_count, VisitID > 2))

write.csv(data, "HospitalReport.csv", row.names = FALSE)
```

Output

Patients Visited More Than Twice:

PatientID	VisitID
101	3
105	4

HospitalReport.csv created successfully.

5. Cricket Statistics Analysis (10 Marks)

A dataset contains **Player Name, Match, Runs, Strike Rate, Venue**.
Write an R program to:

- Read the dataset.
- Create a new data frame containing only players with Strike Rate above 140.
- Calculate average runs venue-wise.
- Export the filtered data into a CSV file.

Analysis

- Read the cricket dataset.
- Create a new data frame with players having **Strike Rate > 140**.
- Calculate **venue-wise average runs**.
- Export the filtered data to a CSV file.

Implementation (R Code)

```
cricket <- read.csv("Cricket.csv")

filtered <- subset(cricket, StrikeRate > 140)

venue_avg <- aggregate(Runs ~ Venue, data = cricket, mean)
print(venue_avg)

write.csv(filtered, "FilteredPlayers.csv", row.names = FALSE)
```

Output

Venue-wise Average Runs:

Chennai 65
Mumbai 72
Delhi 58

FilteredPlayers.csv created successfully.

6. Online Shopping Order Analysis (10 Marks)

Two datasets contain customer details and order details.
Write an R program to:

- Merge the datasets.
- Identify customers with more than three purchases.
- Calculate total purchase amount customer-wise.
- Save the report as an Excel or CSV file.

Analysis

- Read customer and order datasets.
- Merge both datasets.
- Identify customers with **more than 3 purchases**.
- Calculate **total purchase amount** for each customer.
- Save the report as a CSV file.

Implementation (R Code)

```
customer <- read.csv("Customer.csv")
order <- read.csv("Order.csv")

data <- merge(customer, order, by = "CustomerID")

purchase_count <- aggregate(OrderID ~ CustomerID, data, length)
print(subset(purchase_count, OrderID > 3))

total_amount <- aggregate(Amount ~ CustomerID, data, sum)
print(total_amount)

write.csv(total_amount, "ShoppingReport.csv", row.names = FALSE)
```

Output

Customers with More Than 3 Purchases:

```
CustomerID  OrderID
101         4
105         5
```

Total Purchase Amount:

```
CustomerID  Amount
101         25000
102         18000
103         32000
```

ShoppingReport.csv created successfully.

7. Weather Data Transformation (10 Marks)

A weather dataset contains daily temperature values stored as separate columns for each city. Write an R program to:

- Reshape the data using **melt()**.
- Calculate average temperature city-wise.
- Convert the reshaped data back using **dcast()**.
- Write the output into a new CSV file.

Analysis

- Read the weather dataset.
- Reshape the data using melt().
- Calculate **city-wise average temperature**.
- Convert the data back using dcast().
- Save the output as a CSV file.

Implementation (R Code)

```
library(reshape2)

weather <- read.csv("Weather.csv")

long_data <- melt(weather, id.vars = "Day")

avg_temp <- aggregate(value ~ variable, data = long_data, mean)
print(avg_temp)

wide_data <- dcast(long_data, Day ~ variable, value.var = "value")

write.csv(wide_data, "WeatherReport.csv", row.names = FALSE)
```

Output

City-wise Average Temperature:

Chennai 32.5
Mumbai 30.8
Delhi 29.4

WeatherReport.csv created successfully.

8. Banking Transaction Analysis (10 Marks)

Bank maintains two files:

- Customer Details
- Transaction Details

Write an R program to:

- Merge the files.
- Calculate total transaction amount for each customer.
- Identify customers with transactions exceeding ₹1,00,000.
- Export the result.

Analysis

- Read **Customer Details** and **Transaction Details** files.
- Merge both datasets using **CustomerID**.
- Calculate the **total transaction amount** for each customer.
- Identify customers with transactions exceeding **₹1,00,000**.
- Export the result to a CSV file.

Implementation (R Code)

```
customer <- read.csv("CustomerDetails.csv")
transaction <- read.csv("TransactionDetails.csv")

data <- merge(customer, transaction, by = "CustomerID")

total_amount <- aggregate(Amount ~ CustomerID, data, sum)
print(total_amount)

high_transaction <- subset(total_amount, Amount > 100000)
print(high_transaction)

write.csv(high_transaction, "BankReport.csv", row.names = FALSE)
```

Output

Total Transaction Amount:

CustomerID Amount
101 125000
102 85000
103 150000

Customers with Transactions Above ₹1,00,000:

CustomerID Amount
101 125000
103 150000

BankReport.csv created successfully.

9. University Attendance Report (10 Marks)
Attendance data is available in multiple CSV files.
Write an R program to:

- Read all files.
- Combine them into a single data frame.
- Calculate attendance percentage.
- Identify students with attendance below 75%.
- Save the result.

Analysis

- Read multiple attendance CSV files.
- Combine them into a single data frame.
- Calculate **attendance percentage**.
- Identify students with attendance below **75%**.
- Save the result as a CSV file.

Implementation (R Code)

```
files <- list.files(pattern = "*.csv")

attendance <- do.call(rbind, lapply(files, read.csv))

attendance$Percentage <- (attendance$PresentDays / attendance$TotalDays) * 100

low_attendance <- subset(attendance, Percentage < 75)
print(low_attendance)

write.csv(low_attendance, "AttendanceReport.csv", row.names = FALSE)
```

Output

Students with Attendance Below 75%:

StudentID Percentage
102 70
105 68

AttendanceReport.csv created successfully.

10. Movie Rating Analysis (10 Marks)

A dataset contains **Movie Name, Genre, Viewer Rating, Platform**.
Write an R program to:

- Read the dataset.
- Calculate average rating for each genre.
- Reshape the dataset using melt and cast operations.
- Export the transformed dataset.

Analysis

- Read the movie dataset.
- Calculate **average rating** for each genre.
- Reshape the data using melt() and dcast().
- Export the transformed dataset to a CSV file.

Implementation (R Code)

```
library(reshape2)

movie <- read.csv("Movie.csv")

genre_avg <- aggregate(ViewerRating ~ Genre, data = movie, mean)
print(genre_avg)

long_data <- melt(movie, id.vars = c("MovieName", "Genre"))

wide_data <- dcast(long_data, MovieName + Genre ~ variable, value.var = "value")

write.csv(wide_data, "MovieReport.csv", row.names = FALSE)
```

Output

Average Rating by Genre:

Action 4.5
Comedy 4.2
Drama 4.0

MovieReport.csv created successfully.

11. Product Inventory Management (10 Marks)

An inventory system maintains:

- Product Details
- Stock Details
- Supplier Details

Write an R program to:

- Merge all three datasets.
- Identify products with stock less than 20 units.
- Calculate supplier-wise inventory value.
- Save the report.

Analysis

- Read **Product Details**, **Stock Details**, and **Supplier Details**.
- Merge all three datasets.
- Identify products with **stock less than 20 units**.
- Calculate **supplier-wise inventory value**.
- Save the report as a CSV file.

Implementation (R Code)

```
product <- read.csv("ProductDetails.csv")
stock <- read.csv("StockDetails.csv")
supplier <- read.csv("SupplierDetails.csv")

data <- merge(product, stock, by = "ProductID")
data <- merge(data, supplier, by = "SupplierID")

low_stock <- subset(data, Stock < 20)
print(low_stock)

supplier_value <- aggregate(InventoryValue ~ SupplierID, data = data, sum)
print(supplier_value)

write.csv(data, "InventoryReport.csv", row.names = FALSE)
```

Output

Products with Stock Less Than 20:

```
ProductID  ProductName  Stock
101        Laptop       15
104        Keyboard     18
```

Supplier-wise Inventory Value:

```
SupplierID  InventoryValue
S101        250000
S102        180000
```

InventoryReport.csv created successfully.

12. COVID-19 Data Analysis (10 Marks)

A dataset contains **State, Date, Confirmed Cases, Recovered Cases, Deaths**.
Write an R program to:

- Read the dataset.
- Reshape it using melt().
- Calculate state-wise recovery percentage.
- Convert the data back using dcast().
- Export the final data.

Analysis

- Read the COVID-19 dataset.
- Reshape the data using melt().
- Calculate **state-wise recovery percentage**.
- Convert the data back using dcast().
- Export the final data to a CSV file.

Implementation (R Code)

```
library(reshape2)

covid <- read.csv("Covid.csv")

long_data <- melt(covid, id.vars = c("State", "Date"))

covid$RecoveryPercentage <- (covid$RecoveredCases / covid$ConfirmedCases) * 100

print(covid[, c("State", "RecoveryPercentage")])

wide_data <- dcast(long_data, State + Date ~ variable, value.var = "value")

write.csv(wide_data, "CovidReport.csv", row.names = FALSE)
```

Output

State-wise Recovery Percentage:

```
Tamil Nadu    95%
Karnataka     93%
Kerala        97%

CovidReport.csv created successfully.
```

13. Library Management System (10 Marks)

Library maintains:

- Book Details
- Borrower Details
- Issue Details

Write an R program to:

- Merge all datasets.
- Find books issued more than five times.
- Identify overdue books.
- Save the report into a CSV file.

Analysis

- Read **Book Details**, **Borrower Details**, and **Issue Details**.
- Merge all three datasets.
- Find books issued **more than 5 times**.
- Identify overdue books.
- Save the report as a CSV file.

Implementation (R Code)

```
book <- read.csv("BookDetails.csv")
borrower <- read.csv("BorrowerDetails.csv")
issue <- read.csv("IssueDetails.csv")

data <- merge(book, issue, by = "BookID")
data <- merge(data, borrower, by = "BorrowerID")

issued <- aggregate(IssueID ~ BookID, data, length)
print(subset(issued, IssueID > 5))

overdue <- subset(data, DueDate < ReturnDate)
print(overdue)

write.csv(data, "LibraryReport.csv", row.names = FALSE)
```

Output

Books Issued More Than 5 Times:

BookID IssueID
101 6
105 7

Overdue Books:

BookID BorrowerID
101 B201
105 B204

LibraryReport.csv created successfully.

14. E-Commerce Delivery Analysis (10 Marks)

Datasets contain:

- Customer Information
- Order Information
- Delivery Information

Write an R program to:

- Merge all datasets.
- Calculate average delivery time city-wise.
- Find delayed deliveries (>5 days).
- Export the final report.

Analysis

- Read **Customer Information**, **Order Information**, and **Delivery Information**.
- Merge all three datasets.
- Calculate **average delivery time** city-wise.
- Find deliveries taking **more than 5 days**.
- Export the final report as a CSV file.

Implementation (R Code)

```
customer <- read.csv("CustomerInfo.csv")
order <- read.csv("OrderInfo.csv")
delivery <- read.csv("DeliveryInfo.csv")

data <- merge(customer, order, by = "CustomerID")
data <- merge(data, delivery, by = "OrderID")

avg_time <- aggregate(DeliveryDays ~ City, data, mean)
print(avg_time)

delayed <- subset(data, DeliveryDays > 5)
```

```
print(delayed)

write.csv(data, "DeliveryReport.csv", row.names = FALSE)
```

Output

Average Delivery Time:

Chennai 3.5
Mumbai 4.2
Delhi 5.0

Delayed Deliveries:

OrderID	CustomerID	DeliveryDays
101	C201	6
105	C205	7

DeliveryReport.csv created successfully.

15. College Placement Analysis (10 Marks)

Placement data contains:

- Student Details
- Company Details
- Salary Details

Write an R program to:

- Read all datasets.
- Merge them using appropriate keys.
- Calculate department-wise placement percentage.
- Identify the company offering the highest average package.
- Save the final report as a CSV file.

Analysis

- Read **Student Details**, **Company Details**, and **Salary Details**.
- Merge all datasets using appropriate keys.
- Calculate **department-wise placement percentage**.
- Identify the company offering the **highest average package**.
- Save the final report as a CSV file.

Implementation (R Code)

```
student <- read.csv("StudentDetails.csv")
company <- read.csv("CompanyDetails.csv")
salary <- read.csv("SalaryDetails.csv")

data <- merge(student, company, by = "CompanyID")
data <- merge(data, salary, by = "StudentID")

placement <- aggregate(Placed ~ Department, data, mean)
print(placement)

avg_package <- aggregate(Package ~ CompanyName, data, mean)
print(avg_package)
```

```
write.csv(data, "PlacementReport.csv", row.names = FALSE)
```

Output

Department-wise Placement Percentage:

CSE 90%
IT 85%
ECE 80%

Highest Average Package:

Comp anyNa me	Packa ge
TCS	8.5 LPA
Infosys	7.2 LPA

PlacementReport.csv created successfully.

RUBRICS FOR CALCULATION

Analytical Problem Solving					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation